

BÀI 1:

1. GitHub REST API

- **Loại:** RESTful API sử dụng chuẩn HTTP và JSON.
 - **Base URL:** <https://api.github.com>
 - **Authentication:**
 - Cho phép gọi ẩn danh đối với public data (giới hạn 60 requests/giờ).
 - Sử dụng **Bearer Token** qua HTTP header: **Authorization: Bearer <token>** (Personal Access Token hoặc OAuth 2.0 Token) để mở rộng hạn ngạch lên 5.000 requests/giờ.
 - **Versioning:** GitHub trước đây dùng path **/v3**, nhưng hiện nay chuẩn hóa theo **Custom Header: X-GitHub-API-Version: YYYY-MM-DD** (giúp giữ nguyên Base URL mà không làm đứt gãy đường dẫn).
 - **Resource Identifier:** Sử dụng **số nguyên tự tăng (Integer ID)** cho các entity chính (User ID, Repo ID, Issue ID), đồng thời hỗ trợ định danh qua slug name (ví dụ: **/repos/{owner}/{repo}**).
-

2. Spotify Web API

- **Loại:** RESTful API hướng tài nguyên (Resource-Oriented) cho âm nhạc số.
 - **Base URL:** <https://api.spotify.com/v1>
 - **Authentication:** **Bắt buộc 100% OAuth 2.0.**
 - Client phải gửi token qua header: **Authorization: Bearer {access_token}**.
 - Hỗ trợ nhiều luồng: *Authorization Code with PKCE* (dành cho Mobile/SPA), *Client Credentials* (dành cho server-to-server).
 - **Versioning:** Dùng **URI Path Versioning** (**/v1**).
 - **Resource Identifier:** Dùng **chuỗi Base62 alphanumeric 22 ký tự** (ví dụ: **4c0dK2wGLETKBW3PvgPWqT**). Đây là dạng nén của 128-bit UUID, giúp URL ngắn gọn và hoàn toàn ngăn chặn tấn công đoán ID tuần tự (IDOR).
-

3. OpenWeatherMap API

- **Loại:** REST API cung cấp dữ liệu khí tượng.
- **Base URL:** <https://api.openweathermap.org/data/2.5/> (hoặc **/3.0/** cho gói One Call API).
- **Authentication:** Sử dụng **API Key (Secret Token)**.
- Cách phổ biến nhất là truyền qua Query Parameter: **?appid={API_KEY}**.
- Phiên bản mới hơn hỗ trợ truyền qua header: **X-API-Key: {API_KEY}**.
- **Versioning:** Dùng **URI Path Versioning** trực tiếp trên URL path (**/2.5/**, **/3.0/**).
- **Resource Identifier:** Kết hợp giữa **Số nguyên (City ID)** (ví dụ: **id=1581130** cho Hà Nội) và **Tham số địa lý** (**lat={lat}&lon={lon}**).

BÀI 2:

1. Bảng Đối Chiếu

5 Đặc điểm trong Slide 05	Điểm nhấn bổ sung từ JJ Geewax (Ch.1)	Điểm nhấn bổ sung từ James Higginbotham (Ch.1)
1. Hợp đồng rõ ràng (Contract)	Nhấn mạnh tính Predictability (Đoán định được) : Hợp đồng phải nhất quán để dev nhìn 1 endpoint là đoán được cách gọi các endpoint khác.	Nhấn mạnh Outside-In Contract : Hợp đồng phải xuất phát từ nhu cầu người dùng (Job Stories) chứ không bê nguyên cấu trúc bảng database ra.
2. Ẩn cài đặt (Abstraction)	Nhấn mạnh việc tách rời Public Interface khỏi cấu trúc Storage nội bộ để tránh lộ ID tuần tự hay schema DB.	Định nghĩa là Encapsulation : Giữ cho các thay đổi logic nội bộ không làm đứt gãy (break) ứng dụng bên ngoài.
3. Tái cấu trúc (Reusability/Refactoring)	Sử dụng Design Patterns dùng đi dùng lại để tái sử dụng tư duy giải quyết vấn đề phân tán.	Nhấn mạnh Modularization & High Cohesion : Gom nhóm các chức năng có chung mục tiêu nghiệp vụ để dễ tái sử dụng.
4. Độc lập ngôn ngữ & nền tảng	Chuẩn hóa định dạng trao đổi dữ liệu (JSON) và các quy ước kiểu dữ liệu (ISO 8601, UTF-8).	Khẳng định tính Loose Coupling (Khớp nối lỏng) : Client và Server có thể nâng cấp công nghệ độc lập với tốc độ khác nhau.
5. Versionable & Observable	Đưa thêm bài toán Idempotency : API tốt phải an toàn khi client retry do lỗi mạng, không làm sai lệch dữ liệu.	Đưa thêm yếu tố Developer Experience (DX) : Giảm thiểu tối đa tải nhận thức (Cognitive Load) cho lập trình viên tích hợp.

BÀI 3:

```

from flask import Flask, jsonify, request
from uuid import uuid4

app = Flask(__name__)

app.json.sort_keys = False
app.json.ensure_ascii = False

_next = 2
BOOKS = [
    {"id": 1, "title": "Clean Code", "author": "R. Martin", "year": 2008}
]

def find(book_id):
    for book in BOOKS:
        if book["id"] == book_id:
            return book
    return None

# LIST - GET /books
@app.route("/books", methods = ["GET"])
def list_books():
    q = request.args.get("q", "").strip().lower()
    sort_by = request.args.get("sort", "").strip().lower()

    result = BOOKS.copy()

    # (a) Tìm kiếm theo title
    if q:
        result = [b for b in result if q in b.get("title", "").lower()]

    # (b) Sắp xếp theo title
    if sort_by == "title":
        result = sorted(result, key=lambda b: b.get("title", "").lower())

    return jsonify(result), 200

# DETAIL - GET /books/<int:book_id>
@app.route("/books/<int:book_id>", methods = ["GET"])
def get_book(book_id):
    book = find(book_id)
    if book is None:
        return {"error": "Not found"}, 404
    return jsonify(book), 200

# CREATE - POST /books
@app.route("/books", methods = ["POST"])
def create_book():
    global _next
    data = request.get_json(silent = True) or {}
    title = data.get("title")
    author = data.get("author")
    year = data.get("year")

    if not title or not author:

```

```

        return {"error": "Missing title or author"}, 400

# (c) Bắt buộc field year là số >= 1900
if year is None or not isinstance(year, int) or year < 1900:
    return {"error": "Field 'year' must be an integer >= 1900"}, 400

book = {"id": _next, "title": title, "author": author, "year": year}
BOOKS.append(book)
_next += 1
return jsonify(book), 201, {"Location" : f"/books/{book['id']}"}

# UPDATE - PUT, DELETE - DELETE /books/<int:book_id>
@app.route("/books/<int:book_id>", methods = ["PUT", "DELETE"])
def modify_book(book_id):
    book = find(book_id)
    if book is None:
        return {"error": "Not found"}, 404

    if request.method == "PUT":
        data = request.get_json(silent = True) or {}

        if "year" in data:
            year = data["year"]
            if not isinstance(year, int) or year < 1900:
                return {"error": "Field 'year' must be an integer >= 1900"}, 400

        book.update(data)
        return jsonify(book), 200
    else:
        BOOKS.remove(book)
        return "", 204

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001, debug=True)

```